Lập trình mạng · Chương 1

Stream: character stream, charset và framing message

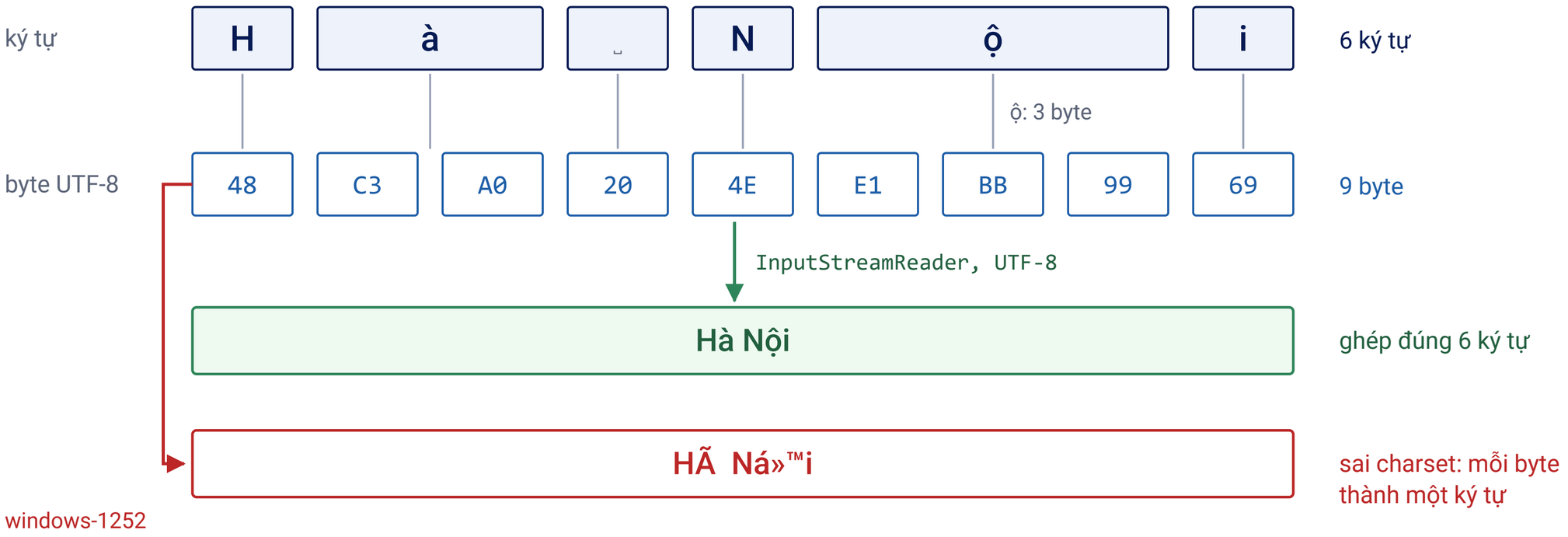
TS. Đặng Ngọc Hùng

Bên nhận cần biết cách đổi byte thành ký tự và ranh giới message

1. Cùng một dãy byte, đọc theo **quy tắc khác nhau** ra chữ khác nhau
2. Byte stream không có ranh giới: hai lần ghi có thể đến thành **một lần đọc**

> **Câu hỏi của phần:** giao thức ứng dụng định nghĩa cách đổi ký tự thành byte và ranh giới message như thế nào?

Charset quy định cách đổi ký tự thành byte, và hai đầu phải dùng cùng charset



Charset quy định cách đổi ký tự ↔ byte; UTF-8 dùng 1 đến 4 byte cho mỗi ký tự.

Character stream (`Reader`, `Writer`) đọc ghi ký tự; `InputStreamReader`, `OutputStreamWriter` chuyển giữa byte và ký tự theo charset.

Framing đánh dấu ranh giới message bằng ký tự phân cách hoặc tiền tố độ dài

Ký tự phân cách (CRLF, như HTTP/1.1)



dữ liệu không được chứa ký tự phân cách; hợp với văn bản: HTTP/1.1, SMTP

Tiền tố độ dài



dữ liệu chứa byte nào cũng được, kể cả byte của CR, LF; hợp với giao thức nhị phân

Ký tự phân cách: đơn giản, hợp với văn bản; dữ liệu không được chứa ký tự đó.

Tiền tố độ dài: dữ liệu chứa byte nào cũng được; bên nhận đọc đủ N byte.

Cùng một dãy byte, đọc bằng hai charset cho hai kết quả

Java

```
var bytes = new ByteArrayOutputStream();  // ở vị trí stream của socket
try (var out = new BufferedWriter(new OutputStreamWriter(bytes, StandardCharsets.UTF_8))) {
    out.write("Hà Nội\n");
}
byte[] received = bytes.toByteArray();
System.out.println(received.length + " bytes: " + HexFormat.ofDelimiter(" ").withUpperCase().formatHex(received));
var utf8 = new InputStreamReader(new ByteArrayInputStream(received), StandardCharsets.UTF_8);
System.out.println("UTF-8:        " + new BufferedReader(utf8).readLine());
var cp1252 = new InputStreamReader(new ByteArrayInputStream(received), Charset.forName("windows-1252"));
System.out.println("windows-1252: " + new BufferedReader(cp1252).readLine());  // SAI charset
```

Kết quả chạy

```
10 bytes: 48 C3 A0 20 4E E1 BB 99 69 0A
UTF-8:        Hà Nội
windows-1252: HÃ  Ná»™i
```

- Cùng 10 byte: UTF-8 ra **Hà Nội**, windows-1252 ra chữ sai, và **không có ngoại lệ**

readLine tách các dòng header của HTTP/1.1 tại CRLF

Java

```
var bytes = new ByteArrayOutputStream();  // các dòng liền nhau trong một khối byte
try (var out = new BufferedWriter(new OutputStreamWriter(bytes, StandardCharsets.US_ASCII))) {
    out.write("GET / HTTP/1.1\r\nHost: ptit.edu.vn\r\nConnection: close\r\n\r\n");
}
var stream = new ByteArrayInputStream(bytes.toByteArray());
try (var in = new BufferedReader(new InputStreamReader(stream, StandardCharsets.US_ASCII))) {
    String line;
    while ((line = in.readLine()) != null && !line.isEmpty()) {  // dòng trống: hết phần header
        System.out.println(line.length() + " chars: " + line);
    }
}
```

Kết quả chạy

```
14 chars: GET / HTTP/1.1
17 chars: Host: ptit.edu.vn
17 chars: Connection: close
```

- Ba dòng trong **cùng một khối byte**; `readLine()` tách đúng tại CRLF và **bỏ CRLF** khỏi dòng trả về
- **Giao thức theo dòng** như HTTP/1.1, SMTP: mỗi dòng kết thúc bằng **CRLF**; dòng trống kết thúc phần header

Tiền tố độ dài cho bên nhận biết trước phải đọc bao nhiêu byte

Java

```java
byte[] data = "chào".getBytes(StandardCharsets.UTF_8);
var bytes = new ByteArrayOutputStream();  // ở vị trí stream của socket
try (var out = new DataOutputStream(bytes)) {
    out.writeInt(data.length);  // tiền tố: 4 byte độ dài
    out.write(data);
}
System.out.println(HexFormat.ofDelimiter(" ").withUpperCase().formatHex(bytes.toByteArray()));
try (var in = new DataInputStream(new ByteArrayInputStream(bytes.toByteArray()))) {
    byte[] message = new byte[in.readInt()];
    in.readFully(message);  // đọc đủ số byte đã báo
    System.out.println(message.length + " bytes: " + new String(message, StandardCharsets.UTF_8));
}
```

Kết quả chạy

```
00 00 00 05 63 68 C3 A0 6F
5 bytes: chào
```

- Độ dài tính bằng **byte**, không phải số ký tự: `chào` có 4 ký tự nhưng 5 byte
- `readFully()` đọc đủ đúng số byte đã báo; stream hết trước thì ném `EOFException`

Luôn chỉ định UTF-8, và tự giới hạn độ dài dòng

1. Không dựa vào charset mặc định: từ Java 18 mặc định là UTF-8, nhưng đầu kia có thể là chương trình khác → **luôn chỉ định UTF-8**
2. `readLine()` **không giới hạn** độ dài dòng: dòng rất dài làm server tốn bộ nhớ → server phải **tự giới hạn**, như web server trả 414 khi request line quá dài

Câu hỏi 1

Chuỗi "Hà Nội" mã hóa bằng UTF-8 dài bao nhiêu byte?

A. 6 byte, vì mỗi ký tự chiếm một byte.

B. 12 byte, vì mỗi ký tự chiếm hai byte.

C. Không xác định được, vì tùy hệ điều hành.

D. 9 byte: à chiếm 2 byte, ộ chiếm 3 byte, mỗi ký tự còn lại 1 byte.

Câu hỏi 1

Chuỗi "Hà Nội" mã hóa bằng UTF-8 dài bao nhiêu byte?

A. 6 byte, vì mỗi ký tự chiếm một byte.

B. 12 byte, vì mỗi ký tự chiếm hai byte.

C. Không xác định được, vì tùy hệ điều hành.

D. 9 byte: à chiếm 2 byte, ộ chiếm 3 byte, mỗi ký tự còn lại 1 byte.

UTF-8 dùng 1 byte cho ký tự ASCII, 2 byte cho à, 3 byte cho ộ: 1 + 2 + 1 + 1 + 3 + 1 = 9 byte. Khi đã chỉ định UTF-8, kết quả không phụ thuộc hệ điều hành.

Câu hỏi 2

Dữ liệu của một giao thức nhị phân có thể chứa byte giá trị 10, chính là byte của ký tự LF. Nên chọn cách framing nào?

A. Tiền tố độ dài: ghi độ dài trước, bên nhận đọc đủ số byte đó.

B. Ký tự phân cách LF, vì readLine đã có sẵn.

C. Không cần framing, vì TCP giữ ranh giới message.

D. Mở một kết nối TCP riêng cho mỗi message là cách duy nhất.

Câu hỏi 2

Dữ liệu của một giao thức nhị phân có thể chứa byte giá trị 10, chính là byte của ký tự LF. Nên chọn cách framing nào?

A. Tiền tố độ dài: ghi độ dài trước, bên nhận đọc đủ số byte đó.

B. Ký tự phân cách LF, vì readLine đã có sẵn.

C. Không cần framing, vì TCP giữ ranh giới message.

D. Mở một kết nối TCP riêng cho mỗi message là cách duy nhất.

Dữ liệu có thể chứa byte của LF, nên LF không dùng làm ký tự phân cách được. Tiền tố độ dài cho phép dữ liệu chứa bất kỳ byte nào. TCP không giữ ranh giới message.

Tổng kết

1. **Charset** quy định ký tự ↔ byte; `InputStreamReader`, `OutputStreamWriter` chuyển đổi theo charset chỉ định

2. **Framing**: ký tự phân cách (LF, `readLine()`) hoặc tiền tố độ dài

3. HTTP/1.1, SMTP là **giao thức theo dòng**: mỗi dòng kết thúc bằng CRLF; luôn chỉ định charset

Từ khóa cần nhớ character stream · Reader · Writer · charset · UTF-8 · framing · ký tự phân cách · tiền tố độ dài · giao thức theo dòng

Mở rộng

Chủ đề nâng cao

- Escape ký tự phân cách trong dữ liệu (như trường có dấu phẩy trong CSV)
- Định dạng TLV (type–length–value)

Bài tập mở rộng

Viết lớp LineFramer đọc từng dòng với giới hạn độ dài tối đa, ném ngoại lệ khi vượt; kiểm thử với dòng tiếng Việt và dòng quá dài.

Tiếp theo

Bài toán ứng dụng: nói HTTP bằng tay với NetCat